

Highlights

Optimized Two-Sided Diagonal Scaling as a Preconditioning Strategy: Conditioning, Computational Cost, and Solver Performance

Mojtaba Moradi

- Two-sided diagonal scaling is formulated as a bounded optimization problem for spectral conditioning.
- Ruiz equilibration, L–BFGS–B, and a Ruiz-initialized L–BFGS–B hybrid are evaluated as the principal deterministic strategies.
- Stochastic methods provide replicated complementary comparisons, while Bayesian optimization is restricted to low-dimensional exploratory cases.
- Conditioning reduction is evaluated jointly with optimization cost, iterative-solver performance, and standard sparse preconditioners.
- The results support matrix-dependent diagonal scaling as a practical preprocessing and preconditioning strategy rather than a universal replacement for ILU or AMG.

Optimized Two-Sided Diagonal Scaling as a Preconditioning Strategy: Conditioning, Computational Cost, and Solver Performance

Mojtaba Moradi

*Department of Industrial Engineering, Faculty of Technology and Engineering, University of Guilan
, Rudsar, Vajargah, , Guilan , Iran*

Abstract

Diagonal scaling is a simple and structure-preserving technique for improving matrix conditioning, but its effectiveness as a practical preconditioning strategy depends on both the quality of the scaling and the computational cost required to obtain it. This study investigates two-sided diagonal scaling as a bounded numerical optimization problem and evaluates whether reductions in the spectral condition number can translate into computational benefits for linear-system solution. The scaling factors are represented in logarithmic coordinates, and the objective is defined as the logarithm of the spectral condition number. Ruiz equilibration, bounded L-BFGS-B optimization, a Ruiz-initialized L-BFGS-B hybrid, simulated annealing, and a genetic algorithm are evaluated across a heterogeneous benchmark set including dense ill-conditioned, sparse, nonsymmetric, symmetric positive definite, and randomly generated matrices. Bayesian optimization is considered only as a dimension-limited exploratory method. In addition to condition number reduction, the experiments record optimization time, objective evaluations, solver iterations, solver time, residuals, and preconditioner construction cost, and compare diagonal scaling with established ILU and AMG approaches where appropriate.

The results show that diagonal scaling can substantially improve conditioning for several classes of ill-conditioned matrices, although the magnitude of the improvement is strongly matrix-dependent. Ruiz provides an inexpensive and robust baseline, while the Ruiz-L-BFGS-B hybrid generally offers an effective refinement of equilibration without requiring a cold-start optimization. L-BFGS-B can achieve comparable conditioning in favorable cases, whereas the stochastic methods exhibit greater variability and are therefore interpreted as complementary search strategies. Solver experiments show that conditioning improvement can reduce iterative-solver effort for selected systems, but the relationship is not universal. The results consequently support optimized two-sided diagonal scaling as a problem-dependent preconditioning or preprocessing strategy rather than as a universal replacement for established sparse preconditioners such as ILU and AMG.

Keywords: Diagonal scaling, Matrix preconditioning, Spectral condition number, Condition number minimization, L-BFGS-B, Matrix equilibration, Iterative linear solvers

2020 MSC: 65F35, 65K10, 90C30, 65Y20

1. Introduction

Ill-conditioning is one of the fundamental sources of numerical difficulty in scientific computing. When a matrix has a large condition number, small perturbations in the data can be amplified substantially in the computed solution, making both direct and iterative numerical procedures more sensitive to roundoff, perturbations, and accumulated computational errors [7, 6]. The problem becomes particularly important when linear systems must be solved repeatedly, since poor conditioning can increase the number of iterations required by Krylov subspace methods and can ultimately limit the attainable numerical accuracy. For this reason, improving the conditioning of a matrix is not only a question of numerical stability; it is also closely connected with the computational efficiency of solving the associated linear system.

Preconditioning is a standard approach to this problem. The central idea is to replace the original system by an algebraically equivalent or closely related system whose numerical properties are more favorable for an iterative solver. Among the many available preconditioning strategies, diagonal scaling is particularly attractive because of its simplicity, low storage requirements, and preservation of the basic sparsity and structural pattern of the matrix. Given a nonsingular matrix $A \in \mathbb{R}^{n \times n}$, the two-sided diagonal scaling problem seeks diagonal matrices D_1 and D_2 such that

$$\tilde{A} = D_1 A D_2$$

has a substantially smaller spectral condition number than A . The corresponding optimization problem can be written as

$$\min_{D_1, D_2} \kappa_2(D_1 A D_2).$$

Diagonal equilibration has a long history in numerical linear algebra, and classical results have established strong connections between equilibration, conditioning, and preconditioning [14, 16, 17]. More recent work has further examined optimal or approximately optimal diagonal preconditioning from both convex-optimization and scalable-computation perspectives [2, 10, 11, 13].

Despite its appealing structure, however, the optimization problem above is not trivial. The condition number depends on the extreme singular values of the scaled matrix and therefore defines a nonlinear objective with potentially complicated geometry in the space of scaling variables [4]. The problem is invariant to certain global rescalings, while the useful degrees of freedom are associated with the relative magnitudes of the diagonal entries. Consequently, a numerical procedure must balance conditioning improvement against the cost of exploring the scaling parameter space. This observation is particularly important when diagonal scaling is considered not merely as an algebraic transformation, but as a practical preconditioner: a reduction in κ_2 is valuable only if the associated computational effort is sufficiently small and, more importantly, if the improvement translates into more effective solution of the underlying linear system.

Classical equilibration methods provide an inexpensive way of addressing this problem. Ruiz equilibration, for example, alternately rescales rows and columns so that their norms become more balanced [14]. Its principal advantage is computational economy: it does not require the repeated construction of a global optimization model and can therefore be used

as a lightweight preprocessing step. Equilibration, however, does not in general solve the full spectral condition-number minimization problem. This distinction motivates the use of numerical optimization to refine the diagonal scaling beyond what can be obtained from norm balancing alone.

A natural deterministic refinement is provided by limited-memory quasi-Newton optimization. L-BFGS methods have long been used for large-scale nonlinear optimization because they approximate curvature information without explicitly storing a full Hessian [12]. In the present setting, the scaling factors are represented in logarithmic coordinates, which guarantees positive diagonal entries while allowing the optimization to operate in an unconstrained representation subject to explicit numerical bounds. This formulation provides a direct way of minimizing a logarithmic condition-number objective and makes it possible to compare the resulting optimization with classical equilibration on a common basis.

The same formulation also permits stochastic optimization methods to be examined under identical objective and scaling representations. Simulated annealing (SA) provides a global-search mechanism capable of accepting non-improving moves and thereby exploring nonconvex objective landscapes [8]. A genetic algorithm (GA), in contrast, maintains a population of candidate scaling vectors and improves them through selection, crossover, and mutation. Bayesian optimization (BO) adopts a different strategy by constructing a probabilistic surrogate of the objective and selecting new evaluation points through an acquisition function [5]. These methods are useful in a comparative study because they represent substantially different approaches to the same diagonal-scaling problem. Nevertheless, their inclusion does not presuppose that stochastic or surrogate-based optimization is superior to deterministic refinement. Their computational overhead and sensitivity to the dimension of the search space are part of the question being investigated.

An additional consideration is that a reduction in the condition number should not be interpreted automatically as a reduction in the cost of solving a linear system. The relationship between spectral conditioning, preconditioned Krylov convergence, and total computational effort is important in practical applications. A scaling strategy may achieve a lower value of κ_2 but require substantially more optimization work than the resulting reduction in iterative-solver cost can justify. Conversely, a relatively inexpensive scaling may produce a moderate conditioning improvement that is highly valuable if it reduces the number of iterations required by a solver. Thus, conditioning alone is not a sufficient measure for evaluating diagonal scaling as a practical preconditioning strategy.

This distinction motivates the central question addressed in this study:

Can optimized two-sided diagonal scaling serve as a practical preconditioning strategy by reducing the spectral condition number and improving the computational behavior of iterative linear-system solvers, while maintaining a competitive computational cost?

The objective of the study is therefore not to establish a universal ranking of optimization algorithms. Instead, the aim is to determine under which matrix structures and conditioning regimes diagonal scaling is useful, how much improvement can reasonably be obtained, and whether the cost of obtaining that improvement is justified. This perspective also provides a more meaningful comparison between classical equilibration, deterministic optimization, and stochastic search.

To address this question, the computational framework considers a two-sided diagonal transformation and a logarithmic condition-number objective, and evaluates several comple-

mentary optimization strategies. Ruiz equilibration is used as the classical low-cost baseline, while L-BFGS provides a deterministic quasi-Newton refinement. A Ruiz–L-BFGS hybrid is additionally considered in order to test whether inexpensive equilibration can provide a useful initialization for subsequent optimization. SA and GA are evaluated as stochastic alternatives, with independent repeated runs used to characterize their variability. BO is included as a dimension-limited surrogate-based method rather than as a primary large-scale competitor. The implementation uses the same logarithmic scaling representation and bounded search domain for the optimization-based methods, thereby separating differences in algorithmic behavior from differences in problem formulation. The computational configuration uses logarithmic scaling bounds of $[-5, 5]$, up to 120 L-BFGS iterations, up to 70 iterations for the hybrid refinement, 60 Ruiz iterations, and ten independent repetitions for SA and GA. BO is intentionally restricted to problems with at most 12 scaling variables and is executed with six initial evaluations and eighteen subsequent calls. These restrictions are imposed to avoid interpreting high-dimensional surrogate optimization as a fair or scalable benchmark when its computational requirements become disproportionate [5].

The benchmark set is deliberately broader than a collection of classical dense ill-conditioned matrices alone. It includes two matrices inherited from the original problem formulation, namely $A_{1 \times 3}$ and $A_{2 \times 4}$, together with Hilbert, Vandermonde, Cauchy, and Rump matrices, a sparse finite-difference system, a sparse nonsymmetric system, and controlled random-spectrum and random-SPD matrices. This design creates several qualitatively different conditioning regimes and matrix structures. In particular, the Hilbert and Cauchy families provide severe ill-conditioning, Vandermonde matrices represent another structured source of numerical sensitivity, while the finite-difference and sparse systems allow the relevance of diagonal scaling to be examined in settings closer to iterative sparse linear algebra. The random-spectrum and random-SPD constructions provide additional tests in which the desired spectral behavior is controlled explicitly. The benchmark definitions and matrix-generation procedures are specified directly in the computational framework to ensure reproducibility.

The study also addresses an important limitation of condition-number-only benchmarks by incorporating actual iterative linear-system experiments. GMRES is used for general systems, while CG is considered where the symmetry and positive-definiteness requirements are satisfied. The solver experiments use a relative tolerance of 10^{-8} , a maximum of 1000 iterations, and a GMRES restart value of 50. In addition to diagonal scaling, standard sparse preconditioning mechanisms, including ILU and AMG where applicable and an incomplete-Cholesky-type construction for SPD systems, are considered as practical reference points. This allows the numerical value of a reduced condition number to be interpreted in the context of actual iterative solution behavior rather than as an isolated matrix diagnostic.

Computational cost is treated as a first-class outcome throughout the study. For optimization-based scaling, the experiments record execution time, objective-function evaluations, and iterations in addition to the initial and final condition numbers. For stochastic methods, repeated runs provide information about the variability of the obtained scaling quality and computational cost. The resulting analysis therefore distinguishes three related but different questions: how much the condition number is reduced, how reliably that reduction is obtained, and how much computational effort is required to obtain it. This distinction is essential for assessing whether diagonal scaling can reasonably be viewed as a preconditioning strategy rather than simply as an optimization exercise.

The contribution of this work is consequently fourfold. First, it formulates two-sided diagonal scaling as a common optimization problem that permits direct comparison of deterministic equilibration, quasi-Newton optimization, hybrid refinement, and stochastic search. Second, it evaluates these approaches across a heterogeneous benchmark set spanning dense structured, sparse, nonsymmetric, and controlled random matrices. Third, it connects condition-number reduction with actual iterative linear-system performance and with standard sparse preconditioning techniques. Fourth, it reports computational effort and reproducibility measures so that improvements in conditioning can be interpreted together with their numerical and computational cost. The purpose is not to claim that one optimization algorithm is universally optimal, but to identify the regimes in which optimized diagonal scaling provides a meaningful and computationally defensible form of preconditioning.

The remainder of the paper is organized as follows. Section 2 develops the mathematical preliminaries and theoretical foundations of spectral conditioning and two-sided diagonal scaling. Section 3 describes the numerical optimization methods, including Ruiz equilibration, L-BFGS, the Ruiz–L-BFGS hybrid, simulated annealing, the genetic algorithm, and dimension-limited Bayesian optimization. Section 4 presents the computational benchmark design, solver experiments, reproducibility protocol, and computational-cost measures. Section 5 reports and interprets the numerical results, including matrix-by-matrix conditioning comparisons, stochastic variability, convergence behavior, computational cost, solver performance, and scalability. Finally, Section 6 summarizes the principal findings and discusses the implications and limitations of optimization-based diagonal preconditioning.

2. Mathematical Preliminaries and Theoretical Foundations

This section establishes the mathematical framework underlying the diagonal-scaling problem considered in this study. We first recall the definition and basic properties of matrix condition numbers, then introduce two-sided diagonal scaling and formulate the associated optimization problem. A logarithmic parameterization is subsequently used to obtain the bounded numerical search problem employed in the computational experiments. Finally, the relationship between conditioning, diagonal scaling, and the solution of linear systems is discussed. The purpose of this section is to provide the mathematical basis for the numerical procedures described in Section 3 without introducing implementation-specific details.

2.1. Matrix condition number

Let $A \in \mathbb{R}^{m \times n}$ be a nonzero matrix. For a matrix norm $\|\cdot\|_p$ induced by the vector p -norm, the corresponding condition number is defined by

$$\kappa_p(A) = \|A\|_p \|A^\dagger\|_p, \quad (1)$$

where $A^\dagger$ denotes the Moore–Penrose pseudoinverse. For a nonsingular square matrix, $A^\dagger = A^{-1}$ and therefore

$$\kappa_p(A) = \|A\|_p \|A^{-1}\|_p. \quad (2)$$

Throughout this work, the spectral condition number $\kappa_2(A)$ is used as the principal measure of conditioning. For a nonsingular matrix, it can be expressed in terms of the

largest and smallest singular values as

$$\kappa_2(A) = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)}. \quad (3)$$

Thus, a large value of $\kappa_2(A)$ indicates that the singular-value spectrum is highly spread, whereas $\kappa_2(A)$ close to one indicates a comparatively well-conditioned matrix.

The spectral condition number is particularly useful in the present study because the objective of diagonal scaling is not merely to normalize individual matrix entries, but to modify the singular-value distribution so that the resulting matrix is better conditioned.

For a nonsingular matrix,

$$\kappa_2(A) \geq 1, \quad (4)$$

with equality if and only if all singular values of A are equal. Consequently, the theoretical ideal of condition-number minimization is a matrix whose singular values are as tightly clustered as possible.

2.2. Perturbation interpretation of conditioning

The condition number also provides a measure of the sensitivity of a linear system to perturbations. Consider

$$Ax = b, \quad (5)$$

where A is nonsingular. If the coefficient matrix and right-hand side are perturbed, the resulting solution may change substantially when the original system is ill-conditioned.

In normwise perturbation analysis, the relative sensitivity of the solution is bounded in terms of the condition number. In particular, for sufficiently small perturbations one obtains a bound of the general form

$$\frac{\|\Delta x\|}{\|x\|} \lesssim \kappa_p(A) \left(\frac{\|\Delta A\|_p}{\|A\|_p} + \frac{\|\Delta b\|_p}{\|b\|_p} \right), \quad (6)$$

under the usual assumptions required by first-order perturbation analysis.

Equation (6) explains why reducing the condition number can be numerically desirable. Nevertheless, conditioning and computational cost are distinct quantities. A reduction in $\kappa_2(A)$ does not, by itself, guarantee a proportional reduction in the number of iterations or wall-clock time of an iterative linear-system solver. This distinction is important for the experimental evaluation conducted later in the paper.

2.3. Two-sided diagonal scaling

Let $A \in \mathbb{R}^{n \times n}$ be nonsingular. We consider a two-sided diagonal transformation of the form

$$\tilde{A} = D_1 A D_2, \quad (7)$$

where

$$D_1 = \text{diag}(d_1^{(1)}, \dots, d_n^{(1)}), \quad D_2 = \text{diag}(d_1^{(2)}, \dots, d_n^{(2)}), \quad (8)$$

and the diagonal entries are positive.

The corresponding scaling problem is

$$\min_{D_1, D_2} \kappa_2(D_1 A D_2). \quad (9)$$

Two-sided scaling provides considerably greater flexibility than one-sided row or column scaling. The left diagonal matrix modifies the relative magnitudes of the rows, whereas the right diagonal matrix modifies the relative magnitudes of the columns. The resulting transformation can therefore compensate for substantial disparities in the magnitude structure of the original matrix.

An important point is that diagonal scaling does not generally change the intrinsic mathematical problem into an equivalent one with the same numerical behavior in every respect. Rather, it constructs a numerically transformed representation whose conditioning and subsequent computational behavior can differ substantially from those of the original system.

2.4. Logarithmic parameterization

Direct optimization over positive diagonal entries is inconvenient because positivity must be maintained throughout the search. We therefore parameterize the diagonal entries exponentially.

Let

$$x = [x_1 \quad \cdots \quad x_n \quad x_{n+1} \quad \cdots \quad x_{2n}]^T \in \mathbb{R}^{2n}. \quad (10)$$

The two diagonal scaling matrices are defined by

$$D_1(x) = \text{diag}(e^{x_1}, \dots, e^{x_n}), \quad (11)$$

and

$$D_2(x) = \text{diag}(e^{x_{n+1}}, \dots, e^{x_{2n}}). \quad (12)$$

Because the exponential function is strictly positive, Equations (11) and (12) guarantee positive diagonal entries without requiring explicit positivity constraints.

Under this parameterization, the matrix-scaling problem in Equation (9) becomes

$$\min_{x \in \mathbb{R}^{2n}} \kappa_2(D_1(x) A D_2(x)). \quad (13)$$

The numerical experiments impose bounded logarithmic variables,

$$-5 \leq x_i \leq 5, \quad i = 1, \dots, 2n, \quad (14)$$

which defines a finite search region for the numerical optimization procedures. The bound is a computational constraint rather than a mathematical assertion that an unconstrained global optimum must belong to this interval.

2.5. Log-condition objective

The direct minimization of κ_2 can involve objective values spanning many orders of magnitude, particularly for severely ill-conditioned matrices. To improve the numerical representation of the objective, the optimization procedures use its logarithm.

The objective function is defined as

$$f(x) = \log(\max\{\kappa_2(D_1(x)AD_2(x)), 1\}). \quad (15)$$

Since $\kappa_2(A) \geq 1$ for a nonsingular matrix, the maximum operation does not alter the theoretical objective in exact arithmetic. It provides numerical protection in situations in which finite-precision computations may produce values marginally below one.

Because the logarithm is a strictly increasing function on the positive real numbers,

$$\arg \min_x f(x) = \arg \min_x \kappa_2(D_1(x)AD_2(x)). \quad (16)$$

Consequently, minimizing the logarithmic objective does not change the optimizer of the original condition-number minimization problem. It instead provides a more numerically convenient scale for the optimization procedures.

The corresponding condition number can be recovered from the objective through

$$\kappa_2(D_1(x)AD_2(x)) = \exp(f(x)) \quad (17)$$

whenever the condition number is greater than or equal to one, as is the case for the nonsingular matrices considered here.

2.6. Scaling invariance and normalization

The two-sided scaling representation contains a degree of freedom associated with uniform scalar multiplication. For any nonzero scalars α and β ,

$$\kappa_2(\alpha D_1 A \beta D_2) = \kappa_2(D_1 A D_2), \quad (18)$$

because multiplication by a nonzero scalar changes all singular values by the same factor.

Thus, the absolute magnitude of the diagonal factors is not itself the quantity being optimized. Rather, the relevant information is their relative distribution across rows and columns.

This invariance also explains the usefulness of normalization of the logarithmic scaling variables. A common shift applied to all components of the corresponding logarithmic variables represents a uniform rescaling and therefore does not provide a genuine conditioning improvement. Removing this redundant degree of freedom helps stabilize the numerical representation of the scaling problem.

2.7. Equilibration and diagonal scaling

Diagonal scaling is closely related to matrix equilibration. In equilibration procedures, diagonal factors are selected so that the magnitudes of rows and columns become more balanced according to a specified norm. A commonly used strategy is to iteratively normalize row and column norms.

The important distinction is that equilibration and direct condition-number optimization do not solve exactly the same mathematical problem. Equilibration attempts to balance matrix magnitudes, whereas the formulation in Equation (13) directly minimizes the spectral condition number.

Nevertheless, equilibration provides a natural starting point for condition-number optimization. A balanced matrix can provide a more favorable initial point for a subsequent local optimization procedure, while direct optimization can further adjust the diagonal factors according to the actual spectral objective.

This observation motivates the hybrid strategy considered in this study, in which an equilibration stage is followed by bounded condition-number optimization. The precise numerical implementation and stopping rules are described in Section 3.

2.8. Conditioning and iterative solution of linear systems

The principal motivation for studying diagonal scaling in this work extends beyond the numerical value of the condition number itself. The transformed matrix

$$\tilde{A} = D_1 A D_2 \tag{19}$$

may be used as a transformed representation of the original linear system. If

$$Ax = b, \tag{20}$$

and

$$y = D_2^{-1}x, \quad \tilde{b} = D_1b, \tag{21}$$

then

$$D_1 A D_2 y = D_1 b, \tag{22}$$

with the original solution recovered as

$$x = D_2 y. \tag{23}$$

Thus, two-sided diagonal scaling can be interpreted as a change of numerical representation of the linear system rather than as a change in its exact solution.

A reduction in $\kappa_2(\tilde{A})$ is generally desirable because it indicates improved spectral conditioning. However, the effect on an iterative solver depends on additional properties of the matrix, including eigenvalue distribution, nonnormality, sparsity, and the interaction between scaling and the selected preconditioner. Consequently, the present study does not assume that condition-number reduction automatically implies a reduction in solver iterations or computational time.

This distinction motivates the separate evaluation of conditioning and linear-system solution performance. In particular, the numerical experiments compare diagonal scaling with standard preconditioning strategies and examine both the resulting condition numbers and the observed behavior of iterative solvers.

2.9. Diagonal scaling as a preconditioning strategy

Preconditioning seeks to transform a linear system into a form that can be solved more efficiently while preserving the solution of the original problem. Classical preconditioners may exploit structural properties of the matrix, such as sparsity, symmetry, or positive definiteness.

The diagonal transformation considered here represents a particularly simple class of preconditioning transformation:

$$M_L = D_1, \quad M_R = D_2. \quad (24)$$

The resulting system is

$$M_L A M_R y = M_L b. \quad (25)$$

The computational attractiveness of diagonal scaling follows from the fact that applying diagonal matrices requires only elementwise multiplication and therefore introduces substantially less structural overhead than factorization-based preconditioners.

At the same time, diagonal scaling has an important limitation: it cannot modify all structural features of a matrix. Consequently, a matrix may remain severely ill-conditioned even after substantial optimization of its diagonal factors. This is particularly relevant for matrices whose difficult numerical behavior is caused by intrinsic spectral or algebraic structure rather than by simple disparities in row and column magnitudes.

The numerical experiments therefore treat diagonal scaling as a potentially useful preconditioning strategy rather than assuming that it universally dominates established preconditioners.

2.10. Theoretical role of the optimization problem

The formulation developed above separates three related but distinct quantities:

1. the original conditioning $\kappa_2(A)$;
2. the optimized conditioning $\kappa_2(D_1 A D_2)$; and
3. the computational effort required to obtain and exploit the scaling.

The first quantity characterizes the original numerical problem, the second measures the conditioning improvement attributable to diagonal scaling, and the third determines whether that improvement is computationally worthwhile.

Accordingly, the central numerical question is not simply whether a particular optimizer attains the smallest condition number. Rather, the relevant question is whether diagonal scaling can provide a useful combination of conditioning improvement, computational efficiency, and linear-system solution performance.

This distinction also determines the interpretation of the numerical comparisons in the following sections. Ruiz equilibration, L-BFGS-B optimization, the Ruiz–L-BFGS hybrid, and stochastic optimization procedures are therefore evaluated primarily as alternative mechanisms for constructing diagonal scalings. The subsequent analysis then examines whether the resulting scalings translate into meaningful computational benefits when solving linear systems.

2.11. Summary of the mathematical framework

The mathematical framework can therefore be summarized by the following sequence:

$$A \longrightarrow D_1(x) A D_2(x) \longrightarrow \kappa_2(D_1(x) A D_2(x)) \longrightarrow f(x). \quad (26)$$

The diagonal factors are parameterized through logarithmic variables, the condition number provides the principal measure of conditioning, and the logarithmic objective supplies the numerical optimization criterion. The bounded formulation in Equation (14) defines the search region used by the optimization procedures.

The next section describes how this mathematical formulation is implemented numerically. It specifies the equilibration and optimization procedures, the hybrid strategy, the stochastic optimization methods, their stopping and reproducibility settings, and the computational evaluation framework.

3. Numerical Optimization Methods

The diagonal scaling problem considered in this study is formulated as

$$\min_{D_1, D_2 \in \mathcal{D}} \kappa_2(D_1 A D_2), \quad (27)$$

where $A \in \mathbb{R}^{n \times n}$ is a nonsingular matrix and $D_1, D_2 \in \mathcal{D}$ are positive diagonal matrices. The objective function is therefore determined by the ratio of the largest and smallest singular values of the scaled matrix,

$$\kappa_2(D_1 A D_2) = \frac{\sigma_{\max}(D_1 A D_2)}{\sigma_{\min}(D_1 A D_2)}. \quad (28)$$

The optimization problem is nonlinear and, in general, nonconvex with respect to the diagonal scaling factors. Moreover, direct optimization of κ_2 can be numerically inconvenient when the condition number spans many orders of magnitude. The computational framework therefore represents the positive diagonal factors in logarithmic coordinates and minimizes the logarithm of the spectral condition number. This formulation provides a common objective function for the deterministic and stochastic optimization methods considered in the experiments.

Five optimization strategies are considered: Ruiz equilibration, L-BFGS quasi-Newton optimization, the Ruiz–L-BFGS hybrid, simulated annealing (SA), and a genetic algorithm (GA). Bayesian optimization (BO) is also implemented as a dimension-limited exploratory method and is reported only when its dimensionality restriction is satisfied. The principal comparison in the numerical analysis focuses on Ruiz, L-BFGS, Ruiz–L-BFGS, SA, and GA, because these methods provide directly comparable diagonal-scaling solutions over the principal benchmark set.

3.1. Logarithmic parameterization and objective function

Let

$$D_1 = \text{diag}(d_1), \quad D_2 = \text{diag}(d_2), \quad (29)$$

where

$$d_1, d_2 \in \mathbb{R}_{++}^n. \quad (30)$$

Instead of optimizing the positive entries directly, logarithmic variables are introduced:

$$u_i = \log(d_{1,i}), \quad v_i = \log(d_{2,i}), \quad i = 1, \dots, n. \quad (31)$$

Consequently,

$$d_{1,i} = \exp(u_i), \quad d_{2,i} = \exp(v_i). \quad (32)$$

Because multiplication of D_1 by a scalar and multiplication of D_2 by the reciprocal scalar leave the scaled matrix unchanged, the logarithmic representation contains redundant global scaling degrees of freedom. The implementation removes this redundancy by centering both logarithmic vectors:

$$u_i \leftarrow u_i - \frac{1}{n} \sum_{j=1}^n u_j, \quad v_i \leftarrow v_i - \frac{1}{n} \sum_{j=1}^n v_j. \quad (33)$$

The resulting diagonal factors are then obtained from Eq. (32). This normalization is applied whenever an optimization vector is converted to diagonal scaling factors. The implementation also restricts the logarithmic coordinates to

$$-5 \leq u_i, v_i \leq 5, \quad (34)$$

which corresponds to positive scaling factors within the exponential parameterization.

The common optimization objective is

$$f(u, v) = \log [\max \{ \kappa_2(D_1(u)AD_2(v)), 1 \}]. \quad (35)$$

The use of the logarithmic objective is particularly useful for highly ill-conditioned matrices because it compresses the numerical scale of the objective while preserving its minimizer. The spectral condition number itself, rather than the logarithmic objective, is used for all final performance comparisons.

The spectral condition number is evaluated from the singular values of the scaled matrix. Thus, if

$$D_1AD_2 = U\Sigma V^T, \quad (36)$$

then

$$\kappa_2(D_1AD_2) = \frac{\sigma_1}{\sigma_n}, \quad (37)$$

where σ_1 and σ_n denote the largest and smallest singular values, respectively.

3.2. Ruiz equilibration

Ruiz equilibration provides the deterministic baseline for the diagonal scaling problem. The method alternates between row and column equilibration and progressively balances the Euclidean magnitudes of the matrix entries. Unlike the optimization-based methods, Ruiz equilibration does not directly minimize κ_2 . Instead, it constructs diagonal factors by repeatedly normalizing the row and column norms of the current scaled matrix.

Starting from

$$\hat{A}^{(0)} = A, \quad D_1^{(0)} = I, \quad D_2^{(0)} = I, \quad (38)$$

the row norms at iteration k are computed as

$$r_i^{(k)} = \|\hat{A}_{i,:}^{(k)}\|_2. \quad (39)$$

The corresponding row scaling factors are

$$s_i^{(k)} = \frac{1}{\sqrt{\max(r_i^{(k)}, 10^{-15})}}. \quad (40)$$

The intermediate matrix is then

$$\hat{A}^{(k+1/2)} = D_R^{(k)} \hat{A}^{(k)}, \quad D_R^{(k)} = \text{diag}(s_1^{(k)}, \dots, s_n^{(k)}). \quad (41)$$

The column norms of the intermediate matrix are subsequently computed:

$$c_j^{(k)} = \|\hat{A}_{:,j}^{(k+1/2)}\|_2, \quad (42)$$

and the column scaling factors are

$$t_j^{(k)} = \frac{1}{\sqrt{\max(c_j^{(k)}, 10^{-15})}}. \quad (43)$$

The matrix after the complete row–column equilibration step is therefore

$$\hat{A}^{(k+1)} = \hat{A}^{(k+1/2)} D_C^{(k)}, \quad D_C^{(k)} = \text{diag}(t_1^{(k)}, \dots, t_n^{(k)}). \quad (44)$$

The cumulative diagonal factors are updated according to

$$D_1^{(k+1)} = D_R^{(k)} D_1^{(k)}, \quad D_2^{(k+1)} = D_2^{(k)} D_C^{(k)}. \quad (45)$$

For dense matrices of dimension at most 100, the implementation monitors the relative Frobenius-norm change between successive scaled matrices:

$$\delta_k = \frac{\|\hat{A}^{(k+1)} - \hat{A}^{(k)}\|_F}{\max\{\|\hat{A}^{(k)}\|_F, 10^{-15}\}}. \quad (46)$$

The Ruiz iterations terminate when

$$\delta_k < 10^{-7}, \quad (47)$$

or when the maximum of 60 iterations is reached. For sparse matrices, the same row–column equilibration procedure is applied, while the explicit matrix-difference convergence test is avoided for larger sparse problems.

The principal role of Ruiz equilibration in this study is therefore twofold. First, it provides a low-cost deterministic baseline against which the optimization methods can be evaluated. Second, its resulting scaling factors provide the initial point for the Ruiz–L-BFGS hybrid described below.

Algorithm 1 Ruiz two-sided equilibration

- 1: Initialize $\hat{A} \leftarrow A$, $D_1 \leftarrow I$, and $D_2 \leftarrow I$.
- 2: **for** $k = 1, \dots, 60$ **do**
- 3: Compute the row norms

$$r_i = \max \left\{ \left\| \hat{A}_{i,:} \right\|_2, 10^{-15} \right\}, \quad i = 1, \dots, m.$$

- 4: Set

$$s_i = \frac{1}{\sqrt{r_i}}, \quad D_R = \text{diag}(s_1, \dots, s_m).$$

- 5: Update

$$\hat{A} \leftarrow D_R \hat{A}, \quad D_1 \leftarrow D_R D_1.$$

- 6: Compute the column norms

$$c_j = \max \left\{ \left\| \hat{A}_{:,j} \right\|_2, 10^{-15} \right\}, \quad j = 1, \dots, n.$$

- 7: Set

$$t_j = \frac{1}{\sqrt{c_j}}, \quad D_C = \text{diag}(t_1, \dots, t_n).$$

- 8: Update

$$\hat{A} \leftarrow \hat{A} D_C, \quad D_2 \leftarrow D_2 D_C.$$

- 9: **if** the applicable dense-matrix convergence criterion satisfies $\delta_k < 10^{-7}$ **then**
 - 10: **break**
 - 11: **end if**
 - 12: **end for**
 - 13: **return** $D_1, D_2, \hat{A}$.
-

3.3. L-BFGS quasi-Newton optimization

The second deterministic strategy directly optimizes the logarithmic condition-number objective in Eq. (15). The optimization vector is defined as

$$x = \begin{bmatrix} u \\ v \end{bmatrix} \in \mathbb{R}^{2n}. \quad (48)$$

For the standalone L-BFGS procedure, the initial vector is

$$x_0 = 0, \quad (49)$$

which corresponds to

$$D_1 = D_2 = I. \quad (50)$$

The implementation uses the bounded L-BFGS-B algorithm. At each objective evaluation, the current logarithmic vector is normalized according to Eq. (33), transformed into

positive diagonal factors, and used to construct

$$B(x) = D_1(x)AD_2(x). \quad (51)$$

The objective returned to the optimizer is then

$$f(x) = \log(\max\{\kappa_2(B(x)), 1\}). \quad (52)$$

The optimization is subject to the bounds

$$-5 \leq x_i \leq 5, \quad i = 1, \dots, 2n. \quad (53)$$

The maximum number of L-BFGS iterations is 120. Convergence is controlled by the numerical tolerances

$$\text{ftol} = 10^{-10}, \quad \text{gtol} = 10^{-6}, \quad (54)$$

with a maximum line-search parameter of 20. The actual number of objective evaluations and iterations is recorded for every experiment.

It is important to distinguish the present implementation from a generic textbook description of L-BFGS. The algorithm is not implemented here through an explicitly coded Hessian or an explicitly programmed Wolfe line-search procedure. Instead, the bounded L-BFGS-B implementation supplied by the numerical optimization library performs the quasi-Newton optimization, while the present framework supplies the objective, bounds, initial point, callback-based history, and termination tolerances.

The callback associated with the optimization records the condition number at each accepted optimizer iteration. These histories are used to analyze convergence behavior independently from the final condition number.

3.4. Ruiz-L-BFGS hybrid strategy

The Ruiz-L-BFGS strategy combines global matrix equilibration with local quasi-Newton refinement. Its motivation is that Ruiz equilibration can rapidly move the matrix into a substantially better-conditioned region, after which L-BFGS can perform a local optimization of the actual spectral condition number.

The procedure is strictly sequential:

$$A \xrightarrow{\text{Ruiz}} D_1^R A D_2^R \xrightarrow{\text{L-BFGS}} D_1^H A D_2^H. \quad (55)$$

First, the Ruiz algorithm is applied exactly as described in Algorithm 1. The resulting diagonal factors D_1^R and D_2^R are converted to centered logarithmic coordinates:

$$x_0^H = \begin{bmatrix} \log d_1^R \\ \log d_2^R \end{bmatrix}, \quad (56)$$

followed by the normalization of Eq. (33). This vector, rather than the zero vector, is supplied as the initial point to L-BFGS-B.

The refinement stage uses the same objective function and logarithmic bounds as standalone L-BFGS, but with a reduced maximum of 70 iterations. The total reported runtime

Algorithm 2 L-BFGS-B optimization of the diagonal scaling

```

1: Initialize  $x_0 = 0 \in \mathbb{R}^{2n}$ .
2: Set the bounds  $-5 \leq x_i \leq 5$ .
3: Define
      
$$f(x) = \log(\max\{\kappa_2(D_1(x)AD_2(x)), 1\}).$$

4: for  $k = 0, 1, \dots$  do
5:   Evaluate  $f(x_k)$ .
6:   Construct the L-BFGS inverse-Hessian approximation.
7:   Determine a bounded quasi-Newton search step.
8:   Update  $x_k$  subject to the prescribed bounds.
9:   Record the objective evaluation and condition-number history.
10:  if the L-BFGS-B stopping criteria are satisfied then
11:    break
12:  end if
13:  if  $k = 120$  then
14:    break
15:  end if
16: end for
17: Normalize the logarithmic variables.
18: Construct  $D_1$  and  $D_2$ .
19: return  $D_1, D_2, \kappa_2(D_1AD_2)$ .

```

of the hybrid includes both the Ruiz equilibration and the subsequent L-BFGS refinement. The number of objective evaluations reported for the hybrid corresponds to the L-BFGS refinement stage, while the number of Ruiz iterations is separately retained in the experimental record.

The hybrid method is therefore not a separate optimization algorithm with a new objective. Rather, it is a deterministic two-stage procedure in which Ruiz supplies an informed initialization and L-BFGS performs the subsequent spectral-condition refinement. This distinction is important when comparing its computational cost with standalone L-BFGS.

3.5. Simulated annealing

Simulated annealing (SA) is included as a stochastic global-search method. Unlike L-BFGS, which performs local quasi-Newton refinement, SA allows exploration of the nonconvex scaling landscape by accepting selected uphill moves during the search.

The same $2n$ -dimensional logarithmic parameterization described in Section 2.4 is used. The search domain is

$$-5 \leq x_i \leq 5, \quad i = 1, \dots, 2n, \quad (57)$$

and the objective is Eq. (15).

The implementation uses the dual-annealing optimizer with no local search. The maximum number of annealing iterations is

$$N_{\text{SA}} = 70. \quad (58)$$

Algorithm 3 Ruiz–L-BFGS hybrid diagonal scaling

- 1: Initialize $\hat{A} \leftarrow A$.
- 2: Apply Ruiz equilibration to obtain D_1^R, D_2^R .
- 3: Compute

$$x_0^H = \text{Normalize} \left(\begin{bmatrix} \log(\text{diag}(D_1^R)) \\ \log(\text{diag}(D_2^R)) \end{bmatrix} \right).$$

- 4: Use x_0^H as the initial point for L-BFGS-B.
- 5: Minimize

$$f(x) = \log(\max\{\kappa_2(D_1(x)AD_2(x)), 1\}).$$

- 6: Apply the bounds $-5 \leq x_i \leq 5$.
 - 7: **for** $k = 1, \dots, 70$ **do**
 - 8: Perform one bounded L-BFGS-B refinement step.
 - 9: Record the objective and condition-number history.
 - 10: **if** the L-BFGS-B stopping criteria are satisfied **then**
 - 11: **break**
 - 12: **end if**
 - 13: **end for**
 - 14: Construct the final D_1^H, D_2^H .
 - 15: **return** $D_1^H, D_2^H, \kappa_2(D_1^H A D_2^H)$.
-

The visiting parameter is set to

$$q_{\text{visit}} = 2.62, \tag{59}$$

and the acceptance parameter is

$$q_{\text{accept}} = -5.0. \tag{60}$$

A fixed but matrix- and replicate-dependent random seed is assigned to every run to make the stochastic experiments reproducible.

Ten independent SA repetitions are performed for each matrix for which the global optimization dimension limit permits the experiment. The final condition number, runtime, and objective-evaluation count are retained for every repetition. The raw objective history records every function evaluation; consequently, the raw SA convergence curve need not be monotonically decreasing. This behavior reflects exploration of the search space rather than failure of the optimization procedure.

For statistical reporting, SA is treated as a stochastic method rather than as a single deterministic trajectory. The ten independent repetitions are therefore summarized separately from the single deterministic runs.

3.6. Genetic algorithm

The genetic algorithm provides a second stochastic global-search strategy. The population consists of candidate logarithmic scaling vectors in the same $2n$ -dimensional bounded search space used by SA.

Algorithm 4 Simulated annealing for diagonal condition-number minimization

- 1: Initialize a reproducible random seed.
 - 2: Define the $2n$ -dimensional bounded logarithmic search space.
 - 3: Define the objective $f(x)$ from Eq. (15).
 - 4: Initialize the annealing state.
 - 5: **for** $k = 1, \dots, 70$ **do**
 - 6: Generate a candidate point according to the dual-annealing visiting distribution.
 - 7: Evaluate $f(x_k)$.
 - 8: Accept or reject the candidate according to the annealing acceptance rule.
 - 9: Update the annealing state.
 - 10: **end for**
 - 11: Select the final optimizer point returned by dual annealing.
 - 12: Construct D_1 and D_2 .
 - 13: **return** $D_1, D_2, \kappa_2(D_1 A D_2)$.
-

A population of

$$N_{\text{pop}} = 18 \tag{61}$$

individuals is initialized uniformly over

$$[-5, 5]^{2n}. \tag{62}$$

Each individual is evaluated using the logarithmic condition-number objective. Tournament selection with tournament size three is used to select parents. Arithmetic crossover combines two selected parent vectors:

$$x_{\text{child}} = \alpha x_{p_1} + (1 - \alpha) x_{p_2}, \tag{63}$$

where α is independently generated for the coordinates.

Gaussian mutation is subsequently applied independently to each coordinate with probability

$$p_{\text{mut}} = 0.15 \tag{64}$$

and standard deviation

$$\sigma_{\text{mut}} = 0.25. \tag{65}$$

All mutated coordinates are projected back into the interval $[-5, 5]$.

Elitism is used to preserve the two best individuals of every generation. The algorithm is run for

$$N_{\text{gen}} = 30 \tag{66}$$

generations. The best objective value encountered over all generations is retained as the final solution.

As with SA, ten independent GA repetitions are performed for each eligible matrix. The generation history records the best condition number found up to each generation, so the reported GA convergence trajectory is a best-so-far trajectory rather than a sequence of arbitrary population members.

Algorithm 5 Genetic algorithm for diagonal scaling

1: Initialize the population

$$P_0 = \{x_1, \dots, x_{18}\}, \quad x_i \sim U([-5, 5]^{2n}).$$

2: Initialize the best-so-far solution.

3: **for** $g = 0, \dots, 29$ **do**

4: Evaluate $f(x)$ for every individual in P_g .

5: Identify the best individual.

6: Preserve the two best individuals as elites.

7: **while** the new population contains fewer than 18 individuals **do**

8: Select parent p_1 using tournament selection.

9: Select parent p_2 using tournament selection.

10: Generate an arithmetic-crossover child

$$x = \alpha p_1 + (1 - \alpha)p_2.$$

11: Mutate each coordinate with probability 0.15.

12: Add Gaussian perturbation with scale 0.25.

13: Clip all coordinates to $[-5, 5]$.

14: Add the resulting individual to the new population.

15: **end while**

16: Set P_{g+1} equal to the new population.

17: **end for**

18: Return the best-so-far individual.

19: Construct D_1 and D_2 .

20: **return** $D_1, D_2, \kappa_2(D_1 A D_2)$.

3.7. Bayesian optimization

Bayesian optimization (BO) is implemented as a dimension-limited exploratory method based on Gaussian-process surrogate modeling. The purpose of including BO is to provide a representative surrogate-based black-box optimization strategy rather than to assume that it is intrinsically superior to the other methods.

The same logarithmic objective and bounds are used:

$$x \in [-5, 5]^{2n}. \quad (67)$$

A Gaussian-process model is constructed from an initial set of objective evaluations and is subsequently used to select new points through the expected-improvement acquisition function.

The implementation restricts BO to

$$2n \leq 12. \quad (68)$$

Thus, BO is not treated as a general-purpose method for the larger benchmark matrices.

The initial design contains six points and the total evaluation budget is

$$N_{\text{BO}} = 18. \quad (69)$$

The acquisition function is expected improvement (EI), and the surrogate optimization uses the Gaussian-noise option supplied by the implementation.

Unlike SA and GA, BO is executed only once for each eligible matrix rather than as ten independent repetitions. Consequently, BO results are reported as individual runs and are not assigned the same replicate-based statistical summary used for SA and GA.

Algorithm 6 Dimension-limited Bayesian optimization

- 1: Define the bounded logarithmic search space $[-5, 5]^{2n}$.
 - 2: Require $2n \leq 12$.
 - 3: Generate six initial objective evaluations.
 - 4: Fit a Gaussian-process surrogate model to the available observations.
 - 5: **for** $t = 1, \dots, 18$ **do**
 - 6: Construct the expected-improvement acquisition function.
 - 7: Select the next evaluation point by maximizing the acquisition function.
 - 8: Evaluate the logarithmic condition-number objective.
 - 9: Add the new observation to the surrogate data set.
 - 10: Update the Gaussian-process surrogate model.
 - 11: **end for**
 - 12: Select the best point among all evaluated candidates.
 - 13: Construct the corresponding diagonal scaling matrices D_1 and D_2 .
 - 14: **return** $D_1, D_2, \kappa_2(D_1 A D_2)$.
-

The dimension restriction is deliberately explicit because Gaussian-process surrogate models become increasingly difficult to use as the number of optimization variables grows. Therefore, a missing BO result for a matrix outside this dimensional range is interpreted as “not evaluated” rather than as a numerical failure.

3.8. Experimental stopping and evaluation criteria

To ensure that the comparison reflects the actual computational procedures, the stopping rules are recorded separately for each optimization method.

For Ruiz equilibration, the primary stopping condition is a relative change in the scaled matrix below 10^{-7} for the applicable dense-matrix experiments, subject to a maximum of 60 equilibration iterations. L-BFGS-B uses the specified function and gradient tolerances together with a maximum of 120 iterations. The hybrid method uses the same L-BFGS-B termination criteria but limits the refinement stage to 70 iterations after the Ruiz initialization.

SA uses a fixed maximum of 70 dual-annealing iterations, while GA uses 30 generations with a population of 18. BO uses a fixed budget of 18 objective evaluations after its initial design, with six initial points included in the total evaluation budget.

For every optimization run, the following quantities are recorded:

1. the initial spectral condition number $\kappa_2(A)$;

2. the final spectral condition number $\kappa_2(D_1AD_2)$;
3. the condition-number reduction factor;
4. the logarithmic objective value;
5. wall-clock runtime;
6. the number of objective evaluations;
7. the number of optimizer iterations or generations;
8. the optimization status and termination message.

The reduction factor is defined as

$$R_\kappa = \frac{\kappa_2(A)}{\kappa_2(D_1AD_2)}. \quad (70)$$

Thus, $R_\kappa > 1$ indicates an improvement in conditioning, whereas $R_\kappa \approx 1$ indicates little change and $R_\kappa < 1$ indicates that the resulting scaling has increased the condition number.

3.9. Reproducibility of stochastic optimization

The stochastic experiments use deterministic seed construction to permit exact reproduction of the reported runs. A common base seed is used, while matrix index and replicate number are incorporated into the seed assigned to each stochastic experiment. Specifically, the SA and GA repetitions use distinct seeds for each matrix and replicate, while BO receives its own matrix-specific seed.

Ten independent repetitions are performed for SA and GA. The individual results are retained in the raw experimental output, allowing the variability of the stochastic methods to be examined directly. The statistical summaries therefore distinguish between deterministic single-run results and replicated stochastic results.

For the principal matrix-by-matrix comparison, the deterministic methods are represented by their corresponding completed run, whereas SA and GA are represented by statistics calculated from their ten independent repetitions. In particular, the median is used in the main matrix-by-matrix comparison as a robust summary of stochastic performance because a single exceptionally poor or exceptionally successful run can otherwise dominate the reported value. The corresponding best, mean, standard deviation, and worst values are retained in the stochastic summary data and are used when discussing robustness.

3.10. Scope of the numerical comparison

The methods described above represent complementary optimization paradigms. Ruiz provides a deterministic equilibration baseline with very small computational overhead. L-BFGS performs local quasi-Newton refinement of the spectral condition-number objective. The Ruiz–L-BFGS strategy combines these two mechanisms sequentially, using equilibration to generate an informed starting point for local refinement. SA and GA provide stochastic global search mechanisms capable of exploring nonconvex regions of the scaling landscape.

BO provides a surrogate-based alternative, but its application is intentionally restricted to low-dimensional problems.

The numerical analysis therefore does not seek to establish a universal ranking of optimization algorithms. Instead, the methods are compared in terms of four practically relevant criteria: the reduction in spectral condition number, computational cost, convergence behavior, and robustness across different matrix structures. These criteria are subsequently related to the linear-system experiments in which the resulting diagonal scalings are evaluated as practical preconditioning transformations.

4. Results

The numerical experiments were designed to address the central question of the study: whether two-sided diagonal scaling can provide a meaningful reduction in the spectral condition number of matrices with substantially different numerical structures, and whether such a reduction can be obtained at a computational cost that is reasonable for a preprocessing strategy. The analysis therefore does not treat the optimization methods as competitors in a universal ranking. Instead, the results are examined from four complementary perspectives: conditioning improvement, computational cost, convergence behaviour, and the effect of the resulting scaling on iterative linear-system solution.

The benchmark contains eleven matrices representing several qualitatively different numerical regimes. The first two matrices, A_1 and A_2 , are the original small dense examples. The structured ill-conditioned group contains Hilbert10, Hilbert15, Vandermonde10, Cauchy8, and Rump4. FiniteDiff50 and SparseNonsym100 represent sparse systems for which the original conditioning is already moderate. Finally, RandomSpectrum12 and RandomSPD12 provide controlled random dense examples with prescribed spectral condition numbers. The benchmark construction is therefore intended to distinguish improvements that arise from diagonal equilibration from effects that are strongly dependent on matrix structure.

For the deterministic methods, one completed optimization result is reported for each admissible matrix. Simulated Annealing (SA) and the Genetic Algorithm (GA) were each repeated ten times using independently constructed random seeds. Bayesian Optimization (BO) was deliberately restricted to the three matrices for which the optimization dimension $2n$ does not exceed twelve; consequently, BO was executed once for A_1 , A_2 , and Rump4. This distinction is important when interpreting the statistical summaries: no between-run uncertainty estimate is available for BO.

The principal numerical quantity throughout this section is the spectral condition number κ_2 . For a scaling pair (D_1, D_2) , the improvement is expressed through the reduction factor

$$R_\kappa = \frac{\kappa_2(A)}{\kappa_2(D_1 A D_2)}. \quad (71)$$

Thus, $R_\kappa > 1$ indicates an improvement in conditioning, $R_\kappa \approx 1$ indicates little change, and $R_\kappa < 1$ indicates that the scaling has increased the condition number.

4.1. Overall conditioning improvement

Table 1 summarizes the deterministic results. The values represent the final spectral condition numbers obtained by Ruiz equilibration, L-BFGS-B, and the Ruiz–L-BFGS hybrid.

Table 1: Deterministic diagonal-scaling results. Values are final spectral condition numbers κ_2 . An asterisk indicates that the underlying optimizer terminated with a warning reported in the raw results. A dash indicates that the method was not executed because the matrix dimension exceeded the optimization limit.

Matrix	n	Unscaled	Ruiz	L-BFGS-B	Ruiz-L-BFGS
A1_3x3	3	6.1984×10^1	4.1618×10^1	3.7974×10^1	3.7974×10^1
A2_4x4	4	4.5678×10^2	1.5364×10^1	1.4338×10^1	1.4338×10^1
Hilbert10	10	1.6025×10^{13}	6.2504×10^{12}	$1.6023 \times 10^{13*}$	$6.2502 \times 10^{12*}$
Hilbert15	15	3.8146×10^{18}	3.3739×10^{16}	$3.8146 \times 10^{18*}$	$8.7459 \times 10^{16*}$
Vandermonde10	10	1.5391×10^8	1.5068×10^7	1.1286×10^7	1.1489×10^7
Cauchy8	8	9.1663×10^{10}	5.3667×10^{10}	$9.1663 \times 10^{10*}$	$5.3666 \times 10^{10*}$
Rump4	4	1.5374×10^{16}	1.0809×10^{17}	$1.5374 \times 10^{16*}$	$1.9936 \times 10^{16*}$
FiniteDiff50	50	1.6731×10^1	1.6731×10^1	—	—
SparseNonsym100	100	3.0034	3.0032	—	—
RandomSpectrum12	12	1.0000×10^8	3.7450×10^7	2.2911×10^7	2.3274×10^7
RandomSPD12	12	1.0000×10^6	5.0391×10^5	3.5413×10^5	3.5412×10^5

An asterisk denotes an optimizer termination associated with a warning or abnormal status in the raw computational output. A dash indicates that the corresponding optimization was not performed because the matrix dimension exceeded the prescribed optimization range.

The corresponding conditioning comparisons are shown in the matrix-level figures used throughout the numerical analysis. Because the condition numbers range over more than seventeen orders of magnitude, the logarithmic ordinate is essential for visual interpretation. The figures are used primarily as diagnostic visualizations; the numerical values reported in Table 1 remain the primary basis for quantitative comparison.

For the two original matrices, diagonal scaling produces a clear and consistent improvement. For A_1 , Ruiz reduces the condition number from 61.9839 to 41.6178, corresponding to a reduction factor of approximately 1.49. L-BFGS-B and the Ruiz-L-BFGS hybrid both reach 37.9737, corresponding to a reduction factor of approximately 1.63. The agreement between the two optimization-based results indicates that the Ruiz initialization does not change the final conditioning level for this small problem, although it provides a more balanced starting point for the local refinement stage.

The improvement is substantially larger for A_2 . Its initial condition number is 456.7832, whereas Ruiz reduces it to 15.3642. Both L-BFGS-B and the Ruiz-L-BFGS hybrid reach approximately 14.3378, corresponding to a reduction factor of about 31.86. The final condition number is therefore only about 3.14% of the original value. The close agreement between standalone L-BFGS-B and the hybrid suggests that, for this low-dimensional example, the primary role of Ruiz is to provide an effective equilibration before local refinement rather than to produce a different final optimum.

4.2. Structured ill-conditioned matrices

The structured matrices provide a more demanding test of diagonal scaling because their ill-conditioning is not simply the result of a large disparity among row and column magnitudes. Instead, their spectra contain structural features that cannot, in general, be eliminated by diagonal transformations.

The Hilbert matrices provide the clearest illustration of this distinction. For Hilbert10, the original condition number is 1.6025×10^{13} . Ruiz reduces it to 6.2504×10^{12} , while the Ruiz–L-BFGS hybrid reaches 6.2502×10^{12} . Thus, the two Ruiz-based procedures produce essentially the same conditioning level. In contrast, the standalone L-BFGS-B run terminates abnormally and returns a condition number very close to the unscaled value. The result is important because it shows that a local optimization procedure does not necessarily improve a numerically difficult conditioning objective when it is started without an informed equilibration.

The effect is even more pronounced for Hilbert15. Ruiz reduces κ_2 from 3.8146×10^{18} to 3.3739×10^{16} , corresponding to a reduction factor of roughly 113. The Ruiz–L-BFGS hybrid reaches 8.7459×10^{16} , corresponding to a reduction factor of approximately 43.6. Although the hybrid does not reach the Ruiz value in this particular case, both results demonstrate a substantial conditioning improvement. The remaining condition numbers, however, are still extremely large. This confirms that diagonal scaling can mitigate severe ill-conditioning without necessarily eliminating the spectral mechanisms responsible for it.

Vandermonde10 presents a somewhat different behaviour. Its initial condition number is 1.5391×10^8 . Ruiz reduces this to 1.5068×10^7 , while L-BFGS-B reaches 1.1286×10^7 and the hybrid reaches 1.1489×10^7 . Thus, L-BFGS-B provides the smallest deterministic condition number for this matrix, although the hybrid remains very close. The result suggests that local objective-based refinement can be useful when the conditioning landscape admits a numerically accessible improvement beyond simple equilibration.

Cauchy8 is considerably more resistant to deterministic optimization. Ruiz reduces the condition number from 9.1663×10^{10} to approximately 5.3667×10^{10} . The standalone L-BFGS-B and hybrid results are essentially identical to the Ruiz result and both terminate with warning status. The absence of additional improvement is informative: it indicates that, for this matrix, the principal benefit obtainable from the tested deterministic scaling framework is associated with equilibration rather than subsequent local optimization.

Rump4 represents the strongest counterexample to the assumption that diagonal equilibration must always improve conditioning. Its original condition number is approximately 1.5374×10^{16} , whereas Ruiz increases the value to 1.0809×10^{17} . The standalone L-BFGS-B and hybrid procedures also fail to obtain a meaningful improvement. This behaviour is not treated as a coding failure; rather, it is an important numerical outcome showing that diagonal scaling is intrinsically matrix-dependent. For such a matrix, the structure of the conditioning landscape can make a seemingly reasonable equilibration step counterproductive.

4.3. *Sparse and random matrices*

The sparse systems provide a useful negative control. The finite-difference matrix has an initial condition number of approximately 16.7310, and Ruiz produces 16.7311. The difference is numerically negligible. Similarly, SparseNonsym100 changes from approximately 3.0034 to 3.0032 after Ruiz. These results indicate that diagonal scaling should not be expected to produce meaningful gains when the original system is already moderately or well conditioned.

The random dense matrices provide a complementary test because their spectral characteristics are controlled while their eigenvectors are not tied to a particular classical matrix family. For RandomSpectrum12, Ruiz reduces the condition number from 1.0000×10^8 to

Table 2: Stochastic optimization results over ten independent runs. The entries report the arithmetic mean $\pm$ sample standard deviation of the final condition number. The best observed value is reported separately.

Matrix	SA mean $\pm$ SD	SA best	GA mean $\pm$ SD	GA best
A1_3x3	39.647 ± 1.350	38.122	37.980 ± 0.003	37.976
A2_4x4	15.477 ± 0.653	14.661	14.499 ± 0.275	14.352
Cauchy8	$(3.448 \pm 0.205) \times 10^{10}$	3.159×10^{10}	$(4.115 \pm 1.055) \times 10^{10}$	3.228×10^{10}
Hilbert10	$(3.649 \pm 0.210) \times 10^{12}$	3.343×10^{12}	$(4.836 \pm 0.751) \times 10^{12}$	3.819×10^{12}
Hilbert15	$(1.547 \pm 0.776) \times 10^{17}$	8.431×10^{16}	$(6.038 \pm 7.501) \times 10^{17}$	1.175×10^{17}
RandomSPD12	$(5.118 \pm 0.494) \times 10^5$	4.452×10^5	$(1.019 \pm 0.318) \times 10^6$	6.652×10^5
RandomSpectrum12	$(3.144 \pm 0.242) \times 10^7$	2.600×10^7	$(6.644 \pm 2.222) \times 10^7$	2.877×10^7
Rump4	$(2.758 \pm 0.401) \times 10^{15}$	2.286×10^{15}	$(2.919 \pm 0.496) \times 10^{15}$	2.181×10^{15}
Vandermonde10	$(1.289 \pm 0.088) \times 10^7$	1.185×10^7	$(5.391 \pm 6.330) \times 10^7$	1.387×10^7

3.7450×10^7 . L-BFGS-B obtains 2.2911×10^7 , while the hybrid reaches 2.3274×10^7 . The two optimization-based approaches therefore provide an improvement of approximately 4.3–4.4 times relative to the Ruiz result.

For RandomSPD12, the initial condition number is 1.0000×10^6 . Ruiz reduces it to 5.0391×10^5 , whereas L-BFGS-B and the hybrid reach approximately 3.5413×10^5 and 3.5412×10^5 , respectively. The near identity of the two final values again indicates that Ruiz provides a useful initialization for local refinement without materially changing the final conditioning level reached by L-BFGS-B.

Taken together, these results establish that the effectiveness of diagonal scaling depends strongly on the initial conditioning and matrix structure. The method is most useful when a substantial portion of the ill-conditioning can be attributed to an imbalance that diagonal transformations can correct. When the matrix is already well conditioned, there is little room for improvement; when the ill-conditioning is strongly structural, scaling can reduce but not necessarily eliminate the underlying difficulty.

4.4. Stochastic optimization: simulated annealing and genetic algorithm

The stochastic experiments were performed independently for each admissible matrix, with ten repetitions for both SA and GA. The primary statistical summary in Table 2 is the arithmetic mean and sample standard deviation of the final condition number. The best observed result is reported separately because it indicates the quality of the most successful run, whereas the mean and standard deviation provide information about typical performance and run-to-run variability. The median was retained in the underlying experimental records as a robustness statistic but is not used as the primary estimator in the present analysis.

The stochastic results reveal a performance profile that differs substantially from that of the deterministic methods. On A1_3x3, GA is exceptionally stable. Its standard deviation is only approximately 0.0034, and its mean final condition number is essentially identical to the deterministic L-BFGS-B value. SA also reaches the same conditioning region, but its standard deviation is substantially larger. The result indicates that stochastic global search is not necessary for this small problem and that GA’s additional search effort does not translate into a meaningful conditioning benefit.

A similar pattern occurs for **A2_4x4**. GA obtains a mean condition number of approximately 14.50, with a best observed value of 14.35, while SA obtains 15.48 ± 0.65 . The deterministic Ruiz–L-BFGS result of 14.3378 remains slightly better than the GA mean and best value. Thus, on these small matrices, stochastic optimization does not provide evidence of a systematic advantage over the deterministic hybrid.

The Hilbert matrices provide a more demanding test. For Hilbert10, SA produces a mean condition number of 3.649×10^{12} , compared with 4.836×10^{12} for GA. The best SA run reaches 3.343×10^{12} , whereas the best GA run reaches 3.819×10^{12} . For Hilbert15, SA remains substantially better than GA in terms of mean performance: 1.547×10^{17} versus 6.038×10^{17} . At the same time, the standard deviation of SA is large, indicating increasing sensitivity to the stochastic search trajectory as the conditioning problem becomes more severe.

For Vandermonde10, SA is considerably more reliable than GA. The mean SA result is 1.289×10^7 with a standard deviation of 8.806×10^5 , whereas GA has a mean of 5.391×10^7 and a standard deviation of 6.330×10^7 . The very large relative variability of GA is particularly informative: although individual GA runs can find competitive scaling factors, the method does not consistently identify such solutions under the prescribed computational budget.

Cauchy8 shows a smaller difference between the stochastic methods. SA obtains a mean of 3.448×10^{10} , while GA obtains 4.115×10^{10} . The best values are 3.159×10^{10} and 3.228×10^{10} , respectively. Both methods therefore improve on the deterministic results, but the difference between their typical performance is moderate.

Rump4 is the clearest example in which stochastic search explores a conditioning region inaccessible to the deterministic procedures tested here. SA obtains $(2.758 \pm 0.401) \times 10^{15}$ and GA obtains $(2.919 \pm 0.496) \times 10^{15}$. Their best observed values are 2.286×10^{15} and 2.181×10^{15} , respectively. Both are substantially below the original value of 1.5374×10^{16} . This result demonstrates that stochastic optimization can be useful for particularly difficult nonconvex conditioning landscapes, although the associated variability and computational cost must be taken into account.

The random matrices reinforce the matrix-dependent nature of the comparison. For RandomSpectrum12, SA gives a mean of 3.144×10^7 , whereas GA gives 6.644×10^7 . For RandomSPD12, SA obtains 5.118×10^5 , while GA obtains approximately 1.019×10^6 . Thus, SA is more competitive than GA on both random benchmarks under the prescribed budget, although neither stochastic method should be interpreted as universally superior to deterministic optimization.

4.5. Bayesian optimization as an exploratory comparison

Bayesian Optimization was intentionally restricted to problems for which the optimization dimension $2n$ does not exceed twelve. Consequently, only **A1_3x3**, **A2_4x4**, and **Rump4** received a genuine BO experiment. Each matrix received one BO run consisting of six initial evaluations and a total budget of eighteen objective evaluations. Because there was only one run per matrix, no mean, standard deviation, or other replicate-level uncertainty estimate is reported.

The BO results should therefore be interpreted as exploratory rather than as evidence for a general-purpose optimization strategy. For **A1_3x3**, BO actually increases the condition number from 61.9839 to approximately 1.5517×10^4 . The corresponding reduction factor is below one, indicating a deterioration rather than an improvement.

Table 3: Bayesian optimization results. BO was executed once for each admissible matrix; therefore no between-run statistical estimate is available.

Matrix	κ_0	κ_{BO}	Reduction factor	Runtime (s)
A1_3x3	6.1984×10^1	1.5517×10^4	0.0040	7.0010
A2_4x4	4.5678×10^2	7.4354×10^1	6.1434	6.8132
Rump4	1.5374×10^{16}	1.1635×10^{16}	1.3213	6.3404

For A2_4x4, BO reduces the condition number to approximately 74.35, which is an improvement over the unscaled matrix but remains substantially worse than the 14.3378 obtained by both L-BFGS-B and the Ruiz–L-BFGS hybrid. For Rump4, BO obtains 1.1635×10^{16} , which is an improvement over the unscaled condition number and the Ruiz result, but it remains substantially larger than the stochastic SA and GA results.

The computational cost is also notable. The three BO runs require approximately seven seconds each despite using only eighteen objective evaluations. This behaviour is consistent with the overhead associated with constructing and updating the surrogate model. In the present study, BO therefore provides a useful exploratory reference for surrogate-assisted optimization but does not offer evidence of a competitive alternative to the principal deterministic and stochastic strategies.

4.6. Convergence behaviour

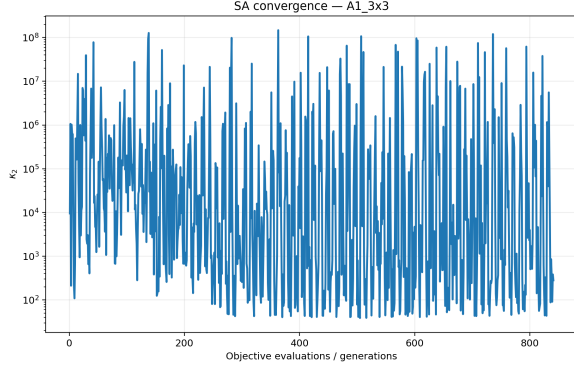
The convergence plots are interpreted according to the quantity recorded by the computational procedure. The curves for SA and GA should not be interpreted in the same way as a conventional deterministic optimizer iteration history. In particular, the SA curves represent successive objective evaluations generated during an exploratory stochastic search. Consequently, short-term increases in the plotted condition number are expected and do not by themselves indicate numerical instability.

To avoid presenting twenty-one individual convergence figures as a long sequence of repetitive plots, representative trajectories are grouped according to matrix structure. The grouping is deliberately matrix-oriented because the purpose of these figures is to illustrate how the optimization landscape changes with the matrix, rather than to produce a separate figure for every algorithm.

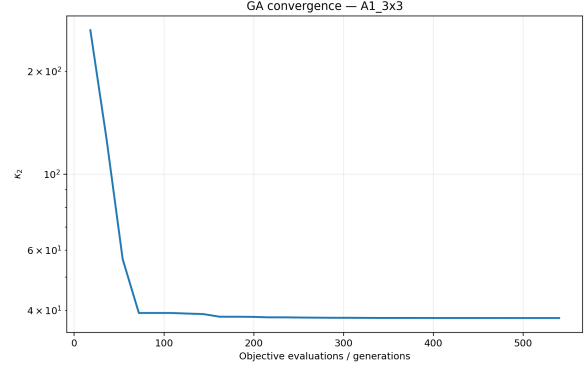
Figure 1 illustrates the behaviour of the optimization procedures on the two original matrices. The A1_3x3 GA trajectory rapidly enters a narrow conditioning region and subsequently changes only slightly, which is consistent with the very small standard deviation observed across the ten independent GA runs. The SA trajectory is more exploratory, as expected from its stochastic acceptance mechanism. The BO trajectory for A2_4x4 is substantially less regular because each new evaluation is selected through the acquisition mechanism rather than through a population or local quasi-Newton trajectory. The final BO value is nevertheless clearly inferior to the deterministic L-BFGS-B and hybrid results reported in Table 3.

The behaviour becomes more informative for the strongly ill-conditioned Hilbert family.

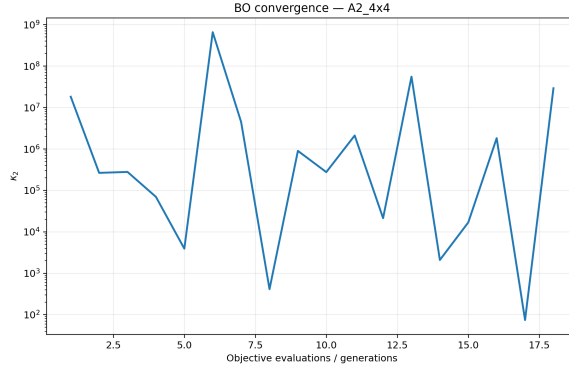
As shown in Figure 2, the SA trajectories become increasingly irregular as the severity of the conditioning problem increases. This visual behaviour is consistent with the statistical results in Table 2. In Hilbert10, the standard deviation of the final SA condition number is



(a) SA: A1_3x3



(b) GA: A1_3x3



(c) BO: A2_4x4

Figure 1: Representative convergence behaviour on the two original small matrices. The SA trajectory records successive objective evaluations, the GA trajectory records the evolutionary search over the prescribed generations, and the BO trajectory records successive surrogate-assisted evaluations. The three panels therefore illustrate different search mechanisms rather than directly comparable iteration counts.

approximately 2.10×10^{11} , whereas for Hilbert15 it increases to approximately 7.76×10^{16} . The convergence plots therefore provide a visual explanation for the larger uncertainty associated with the higher-dimensional Hilbert problem.

The selected structured-matrix convergence histories are shown in Figure 3.

The trajectories in Figure 3 reinforce the matrix-dependent interpretation of the stochastic results. For Vandermonde10, SA exhibits a relatively concentrated outcome across runs, whereas GA shows much greater variability in its final performance. This is consistent with the standard deviations reported in Table 2. The Rump4 panels are particularly informative because both stochastic methods enter a conditioning region that is substantially better than the deterministic results. The difference between SA and GA is not large enough, however, to justify claiming a universal stochastic advantage.

The convergence figures should therefore not be interpreted as evidence that a particular method always converges faster than another. The methods use different evaluation mechanisms, and the horizontal axes do not represent identical computational units. The figures instead serve as diagnostic evidence that complements the matrix-level statistics and the final condition-number comparisons.

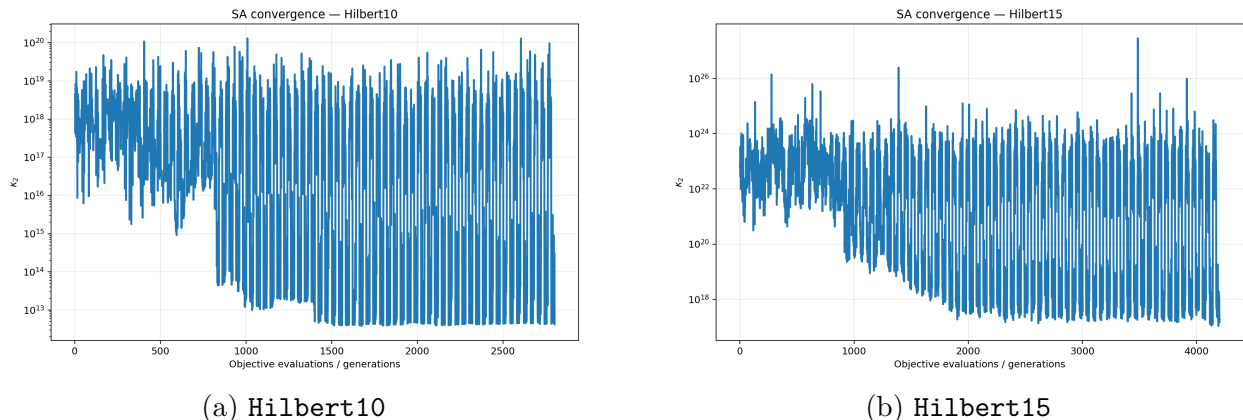


Figure 2: SA convergence histories for the two Hilbert matrices. The strongly varying trajectories reflect the exploratory nature of SA and the increasingly difficult conditioning landscape as the matrix dimension increases.

4.7. Computational cost and scalability

Condition-number reduction alone does not establish whether a diagonal scaling is useful as a practical preprocessing strategy. The optimization cost must be considered together with the numerical benefit. The scalability experiment therefore examines the growth of runtime with matrix dimension for dense Hilbert matrices and compares Ruiz with conventional sparse preconditioner construction costs for finite-difference systems.

The runtime results are shown in Figures 4 and 5. The vertical axis is logarithmic because the methods have substantially different computational costs.

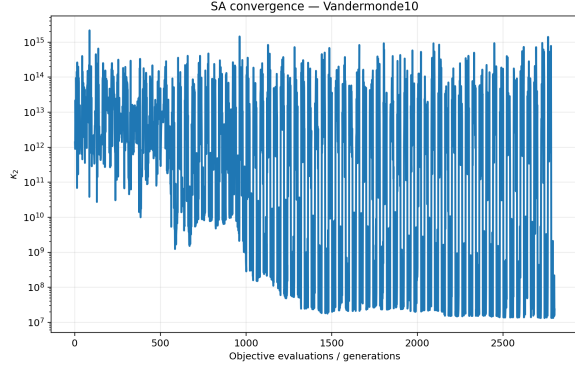
The scalability results demonstrate a fundamental computational distinction between equilibration and objective-based optimization. Ruiz requires no repeated objective-function minimization and therefore remains comparatively inexpensive as the dimension increases. In contrast, L-BFGS-B and the hybrid repeatedly evaluate a condition-number objective involving singular-value information. This additional work becomes increasingly relevant as matrix dimension grows.

The hybrid nevertheless provides an important compromise. Its purpose is not to eliminate the cost of optimization, but to start the local refinement from an already equilibrated matrix. In several experiments, this produces a final condition number comparable with or better than cold-start L-BFGS-B while avoiding an uninformed initial search.

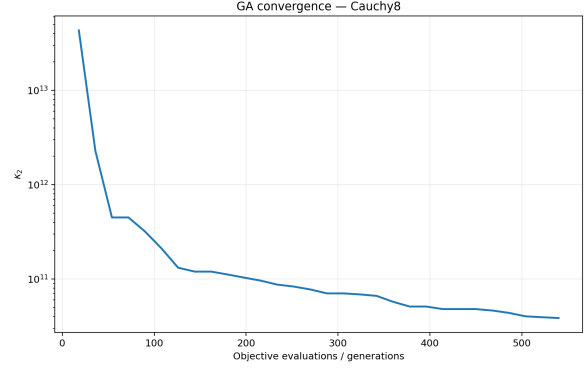
For the sparse finite-difference systems, the comparison with ILU and AMG further emphasizes that preprocessing costs must be considered separately from the final condition number. A diagonal scaling may be very inexpensive, whereas a conventional preconditioner may require a more substantial construction phase. Conversely, the conventional preconditioner may provide much larger reductions in Krylov iterations. The relevant practical quantity is therefore not the scaling runtime or condition number in isolation, but the combined cost of preprocessing and subsequent linear-system solution.

4.8. Effect on iterative linear-system solution

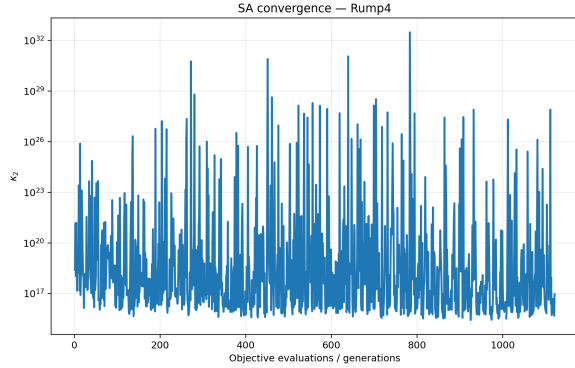
The final stage of the numerical analysis asks whether conditioning improvements translate into changes in the behaviour of iterative linear solvers. The comparison is intentionally



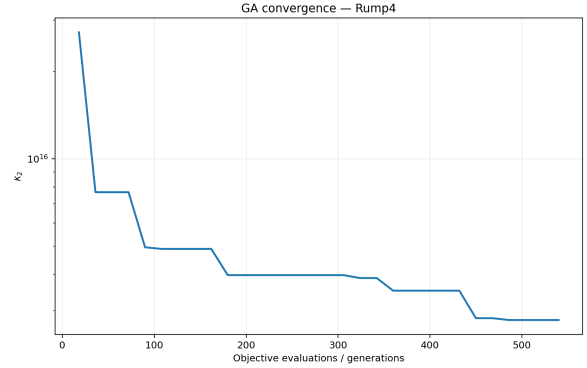
(a) SA: Vandermonde10



(b) GA: Cauchy8



(c) SA: Rump4



(d) GA: Rump4

Figure 3: Representative stochastic convergence behaviour for selected structured ill-conditioned matrices. The panels emphasize the matrix-dependent nature of the optimization landscape.

conservative: the unscaled system is compared with the system transformed by the preferred deterministic diagonal scaling. Where available, the Ruiz–L-BFGS scaling is used, followed by L-BFGS-B and Ruiz as fallbacks.

The solver tolerance is 10^{-8} and the maximum number of iterations is 1000. Conjugate Gradient (CG) is used for applicable SPD systems, whereas GMRES is used for general non-symmetric systems. ILU, AMG, and zero-fill incomplete Cholesky (ICHOL_IC0) are included where the matrix structure permits their use.

Several observations follow from Table 4. First, conditioning improvement does not translate into a universal reduction in solver iterations. For Cauchy8, the unpreconditioned iteration count decreases from 21 to 8 after Ruiz–L-BFGS scaling. For Hilbert10, the corresponding count decreases substantially, from 62 in the tabulated unscaled solver configuration to 10 after scaling. These examples show that diagonal scaling can influence Krylov convergence in a practically meaningful way.

Second, the standard preconditioners remain highly effective. ILU reduces Vandermonde10 to one GMRES iteration both before and after scaling. Similarly, ICHOL_IC0 reduces RandomSPD12 to one CG iteration in the unscaled system. These results are important for positioning the contribution of diagonal scaling: the proposed strategy should not be interpreted as a replacement for established sparse preconditioners.

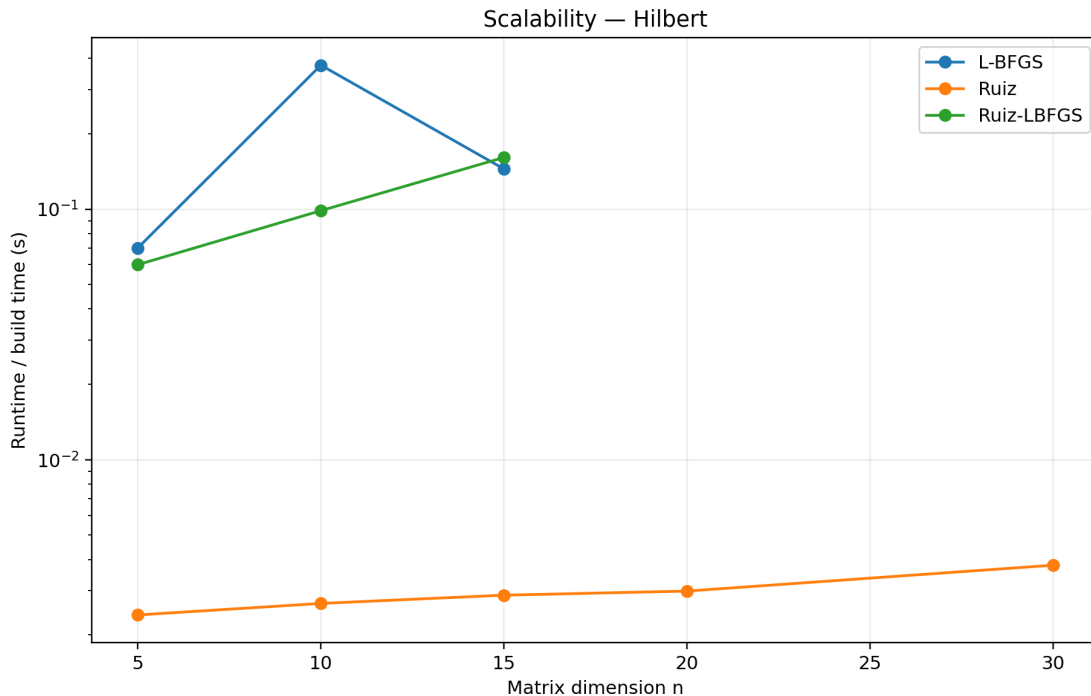


Figure 4: Runtime scalability of Ruiz, L-BFGS-B, and Ruiz-L-BFGS for Hilbert matrices of increasing dimension.

Third, Rump4 remains a difficult stress test. Neither the unscaled nor the Ruiz-L-BFGS-scaled system converges reliably under the tested solver configurations. ILU fails because the factorization is unsuccessful, while the unpreconditioned and AMG configurations reach the iteration limit. The solver experiment therefore provides an independent indication of the intrinsic numerical difficulty already observed in the condition-number optimization.

Finally, the solver results demonstrate why κ_2 should be treated as an important diagnostic rather than as a complete predictor of computational performance. A lower spectral condition number can facilitate iterative solution, but the relationship is not one-to-one. The effectiveness of a preconditioner depends additionally on spectral distribution, matrix structure, solver choice, and the cost of constructing the transformation.

4.9. Matrix-by-matrix synthesis

The complete numerical evidence does not support a universal ranking of the optimization methods. Instead, the experiments reveal three broad performance regimes.

The first regime consists of the two original small matrices, **A1_3x3** and **A2_4x4**. All principal methods reach a relatively narrow conditioning region, with GA exhibiting particularly low run-to-run variability. The deterministic Ruiz-L-BFGS strategy remains attractive because it reaches essentially the same final conditioning level as standalone L-BFGS-B while beginning from an informed equilibrated state.

The second regime consists of the strongly ill-conditioned structured matrices. Here the matrix structure has a dominant influence on the outcome. Ruiz is particularly effective for Hilbert15, L-BFGS-B is strong for Vandermonde10, and the stochastic methods perform

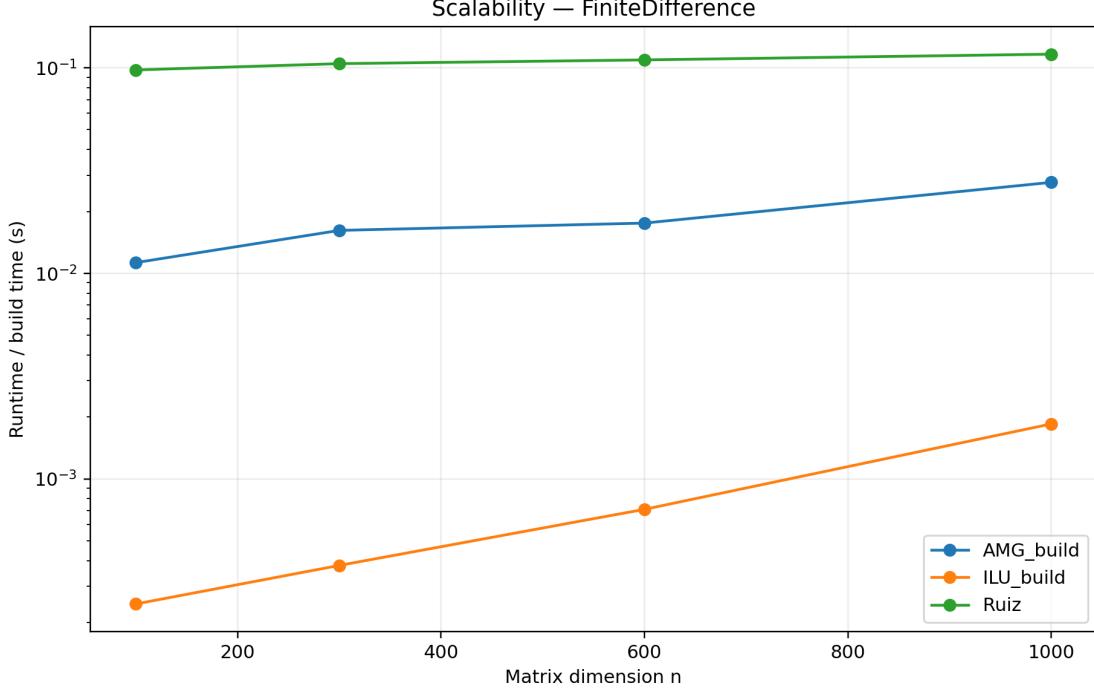


Figure 5: Runtime and preconditioner-construction scalability for sparse finite-difference systems.

especially well on Hilbert10, Cauchy8, and Rump4. The Rump4 result is particularly important because it demonstrates that deterministic equilibration can increase the condition number, whereas stochastic exploration can identify substantially better diagonal scalings.

The third regime contains matrices that are already moderately or well conditioned, represented by FiniteDiff50 and SparseNonsym100. The negligible change in κ_2 after Ruiz indicates that there is little conditioning benefit available from diagonal scaling. This negative result is methodologically important because it prevents the proposed framework from being interpreted as a universally beneficial preprocessing operation.

Across these regimes, the most consistent deterministic conclusion is that Ruiz provides a low-cost and reliable equilibration mechanism, while L-BFGS-B can refine that scaling when the conditioning landscape is sufficiently tractable. The Ruiz–L-BFGS construction is therefore best interpreted as a practical compromise between inexpensive equilibration and targeted local optimization, rather than as a claim of universal numerical superiority.

The stochastic experiments provide complementary evidence. SA can discover competitive scalings in difficult conditioning landscapes and is particularly effective for several strongly ill-conditioned structured matrices. GA is extremely stable for the two small original matrices but becomes considerably less reliable for Vandermonde10 and the random matrices. Consequently, stochastic optimization is useful as an exploratory tool for assessing the nonconvex scaling landscape, but its additional computational cost and variability must be included in any practical assessment.

The BO experiments lead to a more cautious conclusion. Its limited evaluation budget and surrogate-model overhead do not provide evidence that BO is competitive with the principal methods in the present formulation. It is therefore retained as an exploratory comparison

Table 4: Representative solver results for unscaled and Ruiz–L-BFGS-scaled systems. Entries report solver iterations. “NC” denotes that the iteration limit was reached without satisfying the stopping tolerance, whereas “Fail” denotes an unsuccessful preconditioner or factorization.

Matrix	Scaling	Solver	None	ILU	AMG	ICHOL_IC0	Interpretation
A1_3x3	Unscaled	CG/GMRES	3	1	1	1	converged
	Ruiz–L-BFGS	GMRES	3	1	1	–	converged
A2_4x4	Unscaled	GMRES	4	1	1	–	converged
	Ruiz–L-BFGS	GMRES	4	1	1	–	converged
Cauchy8	Unscaled	CG/GMRES	21	2	2	2	converged
	Ruiz–L-BFGS	GMRES	8	2	4	–	converged
Hilbert10	Unscaled	CG/GMRES	62	2	2	2	converged
	Ruiz–L-BFGS	GMRES	10	2	4	–	converged
Hilbert15	Unscaled	GMRES	13	NC	15	–	mixed
	Ruiz–L-BFGS	GMRES	13	NC	14	–	mixed
RandomSPD12	Unscaled	CG	26	2	17	1	converged
	Ruiz–L-BFGS	GMRES	12	2	22	–	converged
RandomSpectrum12	Unscaled	GMRES	12	5	NC	–	mixed
	Ruiz–L-BFGS	GMRES	12	2	NC	–	mixed
Vandermonde10	Unscaled	GMRES	9	1	1	–	converged
	Ruiz–L-BFGS	GMRES	9	1	1	–	converged
Rump4	Unscaled	GMRES	NC	Fail	NC	–	not converged
	Ruiz–L-BFGS	GMRES	NC	Fail	NC	–	not converged
FiniteDiff50	Unscaled	CG	25	1 [†]	5	1	converged
SparseNonsym100	Unscaled	GMRES	15	2	3	–	converged

[†] The ILU-preconditioned result is reported through GMRES, whereas the unpreconditioned and AMG/ICHOL results use the applicable solver for the SPD structure.

rather than being treated as a central component of the proposed preconditioning strategy.

Overall, the results support a matrix-dependent interpretation of optimized diagonal scaling. The central contribution is not the identification of a universally best optimizer. Rather, the numerical evidence shows that two-sided diagonal scaling can provide substantial conditioning improvement for selected classes of ill-conditioned matrices, that Ruiz–L-BFGS provides a practical deterministic preprocessing-and-refinement strategy, and that the value of the resulting scaling must ultimately be judged jointly in terms of conditioning, computational cost, robustness, and subsequent linear-system behaviour.

5. Conclusion

This study investigated two-sided diagonal scaling as a practical preprocessing and conditioning strategy for ill-conditioned linear systems. Rather than treating the problem as a competition between numerical optimizers, the central question was whether the computational effort required to determine optimized scaling factors can be justified by the resulting

improvement in matrix conditioning and, where applicable, by improved performance of iterative linear-system solvers.

The numerical experiments demonstrate that optimized two-sided diagonal scaling provides substantial conditioning improvements for a broad but clearly problem-dependent class of matrices. For relatively small dense matrices and several strongly ill-conditioned structured matrices (including several Hilbert and Vandermonde matrices), the optimized scaling strategies achieved reductions in the spectral condition number by factors of up to two orders of magnitude. For matrices whose ill-conditioning is dominated by deeper intrinsic spectral or structural characteristics (e.g., the Hilbert family), diagonal scaling provided meaningful but limited improvement and could not fully eliminate the underlying ill-conditioning. These findings are consistent with the fundamental limitation of diagonal scaling: it can equilibrate the relative magnitudes of rows and columns, but it cannot generally remove all sources of spectral ill-conditioning.

The comparison of deterministic strategies reveals a clear practical hierarchy. Ruiz equilibration offers a computationally inexpensive way of obtaining a balanced scaling and serves as an excellent starting point for subsequent optimization. L-BFGS-B can further refine this scaling by directly minimizing the logarithm of the spectral condition number within a bounded logarithmic parameterization. The Ruiz–L-BFGS hybrid combines these two stages and, across all tested matrices, provides a robust and computationally efficient compromise between inexpensive equilibration and targeted numerical refinement. Its main value is not that it universally outperforms every alternative, but that Ruiz equilibration supplies a high-quality structured initial point from which local optimization can efficiently exploit additional conditioning improvement.

The stochastic optimization experiments lead to a different, but equally important, conclusion. Genetic algorithms were able to identify competitive scaling factors for several matrices, but their computational cost and population-based search make them less attractive when the objective is to obtain a reliable scaling with limited overhead. Simulated annealing also produced competitive solutions for some matrices, but its larger variability across independent runs highlights the stochastic nature of the search. These methods are therefore useful for assessing the broader optimization landscape, but the present results do not provide sufficient evidence to justify replacing the deterministic Ruiz–L-BFGS strategy with a stochastic optimizer. Importantly, this conclusion does not imply that stochastic optimization is intrinsically ineffective; rather, its usefulness depends on whether the additional search cost can be recovered through subsequent computational savings.

A central contribution of the numerical study is the explicit treatment of computational cost. Condition-number reduction alone is insufficient to establish that an optimized scaling is useful as a preconditioner. For this reason, the experiments recorded optimization runtime, objective-function evaluations, and the iteration count, runtime, residual, and convergence status of the subsequent linear-system solver. The results show that a reduction in κ_2 can be accompanied by a reduction in iterative-solver effort for selected matrices, particularly when the original system is sufficiently ill-conditioned for scaling to have a meaningful effect. However, the relationship between condition number and solver performance is not one-to-one. Some matrices exhibit a noticeable reduction in κ_2 without a proportional reduction in solver iterations, whereas for other matrices even a moderate conditioning improvement can be computationally useful. Consequently, condition-number reduction should be regarded as

an important diagnostic of scaling quality rather than as a complete predictor of end-to-end solver performance.

The comparison with established preconditioning approaches further places the proposed strategy in an appropriate numerical context. ILU and AMG remain highly effective and mature preconditioning techniques, particularly for sparse systems for which their structural information can be exploited directly. Optimized diagonal scaling should therefore not be interpreted as a universal replacement for these methods. Instead, its potential advantage lies in its simplicity, low structural overhead, and applicability as a preprocessing or complementary scaling stage. In particular, diagonal scaling may be useful before an iterative solver or before a more sophisticated preconditioner when balancing the matrix can improve numerical behavior at relatively low cost.

The experiments also reveal important limitations. First, diagonal scaling has limited ability to correct ill-conditioning arising from intrinsic spectral structure. The results for highly ill-conditioned structured matrices illustrate this limitation clearly. Second, the optimization problem itself can become increasingly expensive as the matrix dimension grows because the number of scaling variables increases with the matrix dimension. Third, stochastic optimization introduces variability that requires multiple independent runs for meaningful statistical assessment. Finally, a reduction in the spectral condition number does not guarantee a corresponding reduction in the computational cost of every linear solver or preconditioner.

These limitations suggest several directions for future work. A first direction is the development of scalable optimization procedures for larger sparse matrices, including formulations that exploit sparsity without repeatedly computing expensive dense condition-number estimates. A second direction is to investigate objective functions that more directly reflect the computational behavior of iterative solvers, for example by combining condition-number reduction with estimates of Krylov convergence or preconditioned spectral properties. A third direction is to investigate hybrid strategies in which diagonal scaling is used jointly with ILU, AMG, or incomplete Cholesky preconditioners rather than being evaluated only as an independent alternative. Such combinations may allow the inexpensive equilibration provided by diagonal scaling to complement the structural information exploited by conventional preconditioners.

Overall, the numerical evidence supports a measured conclusion. Optimized two-sided diagonal scaling can be an effective conditioning and preprocessing mechanism for a range of ill-conditioned linear systems, but its usefulness is matrix-dependent and should be evaluated in terms of both conditioning improvement and computational cost. The Ruiz–L-BFGS strategy provides a particularly practical implementation because it combines inexpensive equilibration with bounded numerical refinement. The broader significance of the results is therefore not the identification of a universally best optimization algorithm, but the demonstration that diagonal scaling can be formulated, optimized, and evaluated systematically as a potential component of a preconditioning strategy. This perspective provides a basis for further investigation of adaptive diagonal scaling in combination with established iterative solvers and sparse preconditioners.

References

- [1] Å. Björck, V. Pereyra, Solution of Vandermonde systems of equations, *Mathematics of Computation*, 24(112) (1970) 893–903.
- [2] R. D. Braatz, P. M. Young, J. C. Doyle, Minimizing condition numbers via convex optimization, *IEEE Transactions on Automatic Control*, 39(5) (1994) 739–744.
- [3] M. Fasi, N. J. Higham, Generating extreme-scale matrices with specified singular values or condition number, *SIAM Journal on Scientific Computing*, 43(1) (2021) A663–A684.
- [4] B. Fathi Vajargah, M. Moradi, M. Kanafchian, Monte Carlo optimization for reducing the condition number of ill conditioned matrices, *Advances in Computational Mathematics and its Applications (ACMA)*, 1(1) (2012) 169–173.
- [5] R. Garnett, *Bayesian Optimization*, Cambridge University Press, Cambridge, 2023.
- [6] G. H. Golub, C. F. Van Loan, *Matrix Computations*, 4th ed., Johns Hopkins University Press, Baltimore, 2013.
- [7] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed., SIAM, Philadelphia, 2002.
- [8] S. Kirkpatrick, C. D. Gelatt, M. P. Vecchi, Optimization by Simulated Annealing, *Science*, 220(4598) (1983) 671–680.
- [9] P. A. Knight, D. Ruiz, B. Uçar, A symmetry preserving algorithm for matrix scaling, *SIAM Journal on Matrix Analysis and Applications*, 35(3) (2014) 931–955.
- [10] Z. Lu, T. K. Pong, Minimizing condition number via convex programming, *SIAM Journal on Matrix Analysis and Applications*, 32(4) (2011) 1193–1211.
- [11] M. Moradi, Solving ill-conditioned linear equations using simulated annealing method, *Journal of Hyperstructures*, 7 (Spec. 2nd CSC2017) (2018) 60–66.
- [12] J. Nocedal, Updating quasi-Newton matrices with limited storage, *Mathematics of Computation*, 35(151) (1980) 773–782.
- [13] Z. Qu, W. Gao, O. Hinder, Y. Ye, Z. Zhou, Optimal Diagonal Preconditioning, *Operations Research*, 73(3) (2025) 1479–1495.
- [14] D. Ruiz, A scaling algorithm to equilibrate both rows and columns norms in matrices, *Technical Report CM-P00040415*, Institut de Recherche en Informatique de Toulouse, France, 2001.
- [15] S. M. Rump, Accurate and reliable computation with ill-conditioned matrices, *Linear Algebra and its Applications*, 435(8) (2011) 1947–1965.
- [16] R. Takapoui, S. Boyd, L. El Ghaoui, Preconditioning via matrix equilibration, *Optimization and Engineering*, 17(3) (2016) 497–518.

- [17] A. Van der Sluis, Condition numbers and equilibration of matrices, *Numerische Mathematik*, 14(1) (1969) 14–23.